

ANTHROPIC 工程博客

用 LLM 守护 源代码安全

把 Claude 变成一支随时上岗的安全团队：
从威胁建模到补丁验证的六步流程

来自前线

发现已经不是瓶颈，验证和修复才是

1,596

已披露的开源漏洞
截至 2026 年 5 月 22 日

97

据我们所知
已经被修复的数量

~90%

有完善威胁模型的团队
能达到的可利用率

"扫描已经可以轻松并行，瓶颈已经转移到验证、分诊和打补丁。"

防守方的循环：投入一次，反复扫描

The defender's loop

Discovery is easy to parallelize. The bottleneck is everything after.

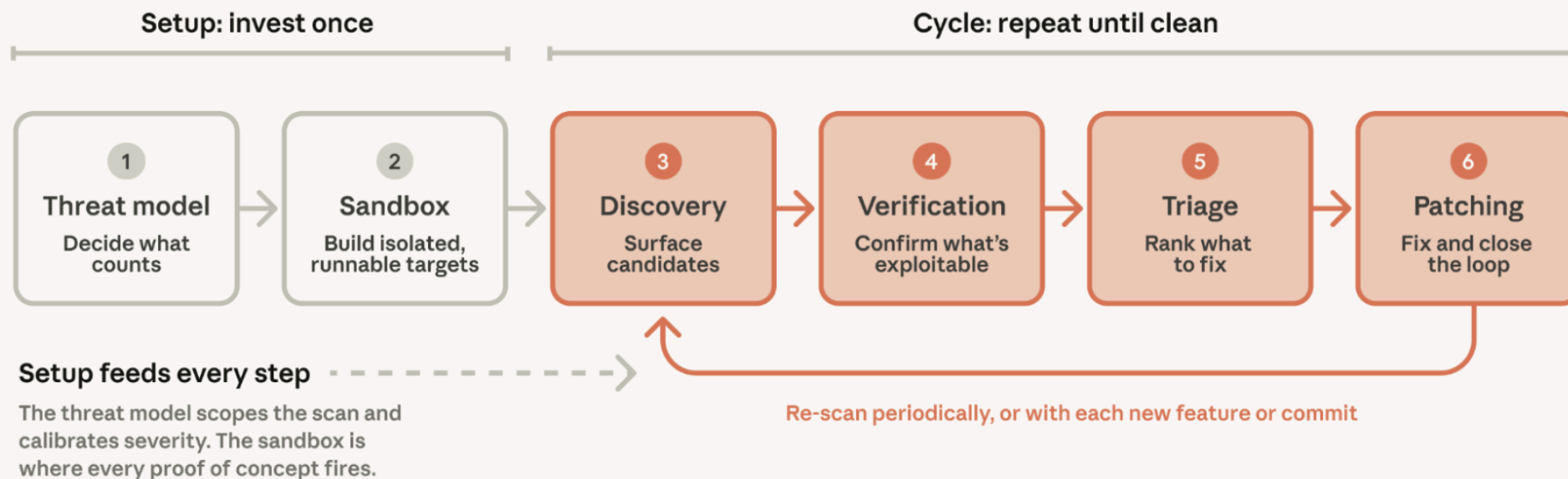


Figure 1 • 一次性投入 = 威胁模型 + 沙箱；循环 = 发现 → 验证 → 分诊 → 修复

PART 1 • SETUP

一次性投入： 把扫描的“地基”打稳

1 Threat Model • 决定"算什么漏洞"

在动手扫描之前，先把"重要的东西"写清楚

如果不先定义什么算漏洞，模型就会在**任何看起来可疑**的地方报警。

- 从现有代码、文档和 **CVE** 历史里 **bootstrap** 出第一版
- 用 Shostack 四问框架做团队访谈
- 产出 **THREAT_MODEL.md**，明确**信任边界**和**关键资产**

关键发现

有完善威胁模型的团队，发现的漏洞可利用率
达到 **~90%**。

"过去被人利用过的姿势" —— 比"帮我找漏洞"是更省力的**作弊码**。

2 Sandbox · 让 PoC 能跑、能炸、但炸不到生产

既是安全护栏，也是"可利用性"的实验台

容器

用于代码静态分析，启动快、并行多。

microVM / 完整 VM

用于跑真正的 PoC，可以触发崩溃、写文件、提权。

网络隔离

初始化完成后切断公网，只保留必要的 API 连接。

钉住依赖版本、构建产物和 `commit SHA`，确保每次重放的结果可复现。

PART 2 • THE CYCLE

循环四步：

发现 • 验证 • 分诊 • 修复

3

Discovery • 把候选漏洞捞上来

不要写检查清单，要给上下文

✓ 这样做

- 把架构文档、威胁模型一起喂进去
- 用"为什么"和"什么"的开放式提问
- 把代码库切分成多个 focus area
- **并行**多个 agent，避免大家都收敛到浅层 bug
- 用结构化字段返回结果

✗ 避免

- 写一份大而全的 checklist 让模型对照
- 用单线 agent 串行扫整个 repo
- 只让模型输出自由格式的描述
- 把发现 (discovery) 和验证 (verification) 揉在一起

4 Verification • 证伪，而不是证实

用一个"全新环境"的 AGENT，专门来挑刺

- 验证 agent 要在**独立的 fresh** 环境里跑
- 提示词里明确要求：**试图推翻**这个发现
- 有条件就强制生成 **PoC** 代码
- 多个独立验证 agent 做**多数投票**，提升精度

效果

通过对抗式验证，团队把"**不可利用**"的误报数量降低了 **~50%**。

关键：发现和验证**目标不同**，要用不同的 agent、不同的提示、不同的环境。

5 Triage • 先去重，再排序

按"根因"去重，不要按"症状"去重

🔽 先用确定性规则过滤

- 同一文件
- 同一漏洞类别
- 代码位置邻近

⚖️ 再按严重程度打分

- 可达性 (reachability)
- 攻击者可控程度
- 触发前置条件
- 是否需要鉴权
- 影响范围 (scope)

6 Patching · 先写失败的测试，再写补丁

闭环的最后一公里，也是最容易翻车的一段

验收一个补丁，至少要过四关：

- 项目构建成功
- 原本的 **PoC 不再触发**
- 回归测试全部通过
- 再让 agent 做一次**对抗式复查**

两层"找变种"

层一：同一段模式在仓库中其它位置的复制

层二：同一**漏洞类别**在其它地方的不同实现

不要顺手做不相关的重构。补丁越小越好。

几个反直觉的发现

首次扫描数量最多

之后每次重扫，新发现的数量会下降，但**复杂度上升**。要提前设定停扫标准。

同一份代码也会出新结果

模型是随机的。即使代码没变，多跑几次仍会冒出新发现——这是特性，不是 bug。

模型懂代码，不懂"我们"

它能看懂逻辑，但不懂业务上哪些路径才**真的重要**。这就是威胁模型存在的意义。

人类还得在环里

架构约束、服务依赖、上下游影响——这些 context-dependent 的判断，目前仍需要人类工程师来做。

动手试试

开箱即用的五个 Skills

threat-model

建立威胁模型

vuln-scan

并行扫描漏洞

triage

去重与排序

patch

生成与验证补丁

customize

按项目定制

发现已经廉价，**"把它修对"** 才是新的护城河。

参考实现： [defending-code-reference-harness](#) • 也可通过 Claude Security 与 GitHub Action 集成